

Simple and Multiple Linear Regression in R

Valentina Lorenzoni, Chiara Seghieri

Sant'Anna School of Advanced Studies, Pisa

`valentina.lorenzoni@santannapisa.it`; `chiara.seghieri@santannapisa.it`

Applied Statistics Lab for the Social Sciences

A.A. 2025-2026

Contents

1	Overview on R-programming language	1
1.1	Download R and RStudio	1
1.2	Syntax tips	2
2	Introduction	2
3	Installation of Packages	2
3.1	Loading the Libraries	2
4	Loading and Exploring the Dataset	3
4.1	Preparing the session	3
4.2	Loading the Dataset	3
4.3	Pre-processing data: cleaning and preparation	5
4.4	Exploratory Data Analysis	7

1 Overview on R-programming language

R is a programming language used for statistical computing and data visualization. It is widely adopted in fields such as data mining, bioinformatics, and data analysis. The core R language is enhanced by a large number of extension packages, which include reusable code, documentation, and example data. R software is open-source and freely available. It is licensed under the GNU General Public License and is part of the GNU Project. It is primarily written in C, Fortran, and R itself. Precompiled executables are available for various operating systems. As an interpreted language, R has a native command-line interface. Additionally, several third-party graphical user interfaces are available, such as RStudio, an integrated development environment, which is the one we adopted in this course.

1.1 Download R and RStudio

Both R and the RStudio environment are free to download at the following links:

- R version 4.5.2: <https://www.r-project.org>
- RStudio: <https://posit.co/downloads/>

Both are supported by different platforms: Windows, Macintosh, Linux and other UNIX versions.

1.2 Syntax tips

The `help()` function in R is used to access the documentation and detailed information about specific functions, datasets, or packages. It allows users to look up descriptions, usage, arguments, examples, and other relevant details about an R function or object directly within the R environment.

Listing 1: *help* function

```
1 # Obtain information about cbind()
2 help(cbind)
```

This will open the help page for the function *cbind()*, displaying details such as usage, arguments, return value, and examples.

As shown in ??, in the R environment, it is possible to use the symbol *#* to add a comment. Well-commented code is essential as it enhances both shareability and comprehensibility, especially in collaborative work environments.

2 Introduction

In this exercise, we will use the data *mensa_esercitazione.dta* to have some preliminary introduction of the use of R to load, explore, and perform basic descriptive analysis in R to have a preliminary check of data and to then conduct a comprehensive regression analysis using the R programming language.

The dataset we will use is related to cashier transactions at one of the canteens serving Pisa University. The dataset contains a sample of all the records of consumed meals, and collected data about date and hours of meals, the number and type of dishes, users' IDs, users' type, calories, etc.. The goal is to model the relationship between calories of the meal consumed and different independent variables such as sex, age, registration year, type of course, etc, using both simple and multiple linear regression. The database is part of a project and made available for the practical session of the course.

3 Installation of Packages

Before proceeding with the analysis, ensure the required R packages are installed. If not, run the following commands to install the necessary libraries.

Listing 2: Installation of Required Packages

```
1 # Install the necessary packages
2 install.packages("haven") # Import file in different format (i.e.,Excel,Stata,
   SAS)
3 install.packages("ggplot2")      # For plotting
4 install.packages("dplyr")        # For data manipulation
5 install.packages("summarytools") # For data overview
6 install.packages("corrplot")     # For data correlation
7 install.packages("car")          # For regression diagnostics
8 install.packages("tidyverse")    # Data processing
9 install.packages("psych")        # Pairing variables
```

3.1 Loading the Libraries

Once the packages are installed, you can load them as follows:

Listing 3: Loading Libraries

```
1 # Load the necessary libraries
2 library(haven)
3 library(ggplot2)
4 library(dplyr)
5 library(summarytools)
6 library(corrplot)
7 library(car)
8 library(tidyverse)
9 library(psych)
```

4 Loading and Exploring the Dataset

In this section, we will import the dataset into RStudio, perform a quick exploratory analysis, and clean the data if necessary. Exploratory data analysis (EDA) is crucial, as it helps uncover underlying patterns,

detect anomalies, and make sense of the dataset's structure. Even more critical is the data preparation and cleaning process, which ensures the dataset is free from inconsistencies, missing values, and errors. A well-prepared dataset is the foundation of any reliable statistical analysis or predictive model, as it directly impacts the accuracy and validity of the results.

4.1 Preparing the session

Before loading data, make sure to set the working directory. Specifying the working directory is a convenient way of saving time and organizing your work, as if you do that at the beginning of the session you don't need to specify the path when importing data, when saving figures, or files created in the session. You can set the working directory either using R command or the menu on the top of the R window.

Listing 4: Setting the working directory

```
1 # Set the directory of work
2 setwd("specify the path of the folder to be used using the forward slash")
```

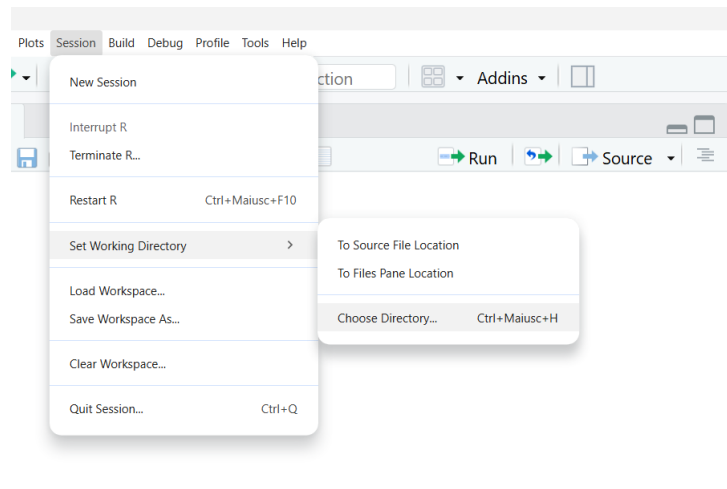


Figure 1: Setting working directory using the menu.

4.2 Loading the Dataset

We load the dataset using the `read_dta` function.

Listing 5: Loading the Dataset

```
1 # Load the dataset
2 dataset <- read_dta("mensa_esercitazione.dta")
3
4 # View the first few rows of the dataset and its dimension RxC
5 head(dataset)
6
7 # View the "dimensions" of data in terms of rows and columns
8 dim(dataset)
9
10 [1] 20000 17
11
12 # Show the internal structure of each column
13 str(dataset)
```

	registration_year	canteen_code	meal_date	sex	birth_place	zip_birth_place	code_birt_place	ID	meal_type	user_type	meal_hour	on_time	course	meal_year	calories	age	study_department
1	2	2	20140325	F	Milan	20121	MI	39735683	4	1	12	1	2	2014	506	26	SCIENZE AGRARIE
2	3	1	20120209	M	Sant	98076	ME	3995592	3	1	12	1	1	2014	314	23	FILOLOGIA
3	1	1	20110304	F	RIO D	NA		6574360	1	1	12	1	2	2014	449	24	INGEGNERIA CIVILE
4	1	3	20140527	M	Nazar	NA		39780728	4	1	13	2	2	2014	765	25	INGEGNERIA INFORM

Table 1: Table of the first four rows of the database, provided by the command `head(dataset)`.

The dataset, as shown in Table ??, provides detailed information on users' characteristics (in terms of both demographics and university career) and details about meals consumed across 20000 accesses over different days between 2010 and 2014. The dataset is a sample of a larger dataset covering multiple years and reporting detailed information on the dishes selected. It contains a total of 20000 rows and 17 columns.

Dataset Variables:

- **registration_year:** Registration year
- **canteen_code:** Identifier of the canteen
- **meal_date:** Date of meal
- **sex:** Sex
- **birth_place:** Place of birth (city)
- **zip_birth_place:** ZIP code for birth place
- **code_birth_place:** County code for birth place
- **ID:** Person unique identifier
- **meal_type:** type of meal
- **user_type:** Type of user
- **meal_hour:** Hour of the meal (HH)
- **on_track:** Type of registration
- **course:** Type of course
- **meal_year:** Year of the meal
- **calories:** Meal's calories (RESPONSE VARIABLE)
- **age:** Age (years)
- **study_department:** Study department

Registration year:

- 1: First year
- 2: Second year
- 3: Third year
- ..

Sex:

- M: Male
- F: Female

Meal type:

- 1: Light meal type 1 (pasta or soup+vegetables+fruit or cake)
- 2: Light meal type 2 (main course+vegetables+fruit or cake)
- 3: Pizza or sandwich
- 4: Full meal (pasta or soup+main course+vegetables+fruit or cake)

User type:

- 1: Students
- 2: Foreigns (not Erasmus)

Type of registration:

- 1: On track
- 2: Not on track

Course:

- 1: Bachelor
- 2: Master degree
- 4: Degree
- 5: PhD

4.3 Pre-processing data: cleaning and preparation

We need to check for missing values, redundant columns, and duplicate rows to ensure the data is ready for analysis. Our database presents a few missing or unknown values for some variables and we could remove the associated rows, while for some variables missing identified a particular type of users. Moreover, some variables are redundant (may cause **multicollinearity**) and we could remove, while other potentially important variables should be created using existing ones. Finally, we remove duplicated rows, which are redundant for the analysis.

Listing 6: Pre-processing dataset

```
1
2
3 # Check for missing values in the dataset
4 sum(is.na(dataset))
5 [1] 443
6
7 # Show the summary of each column and look at missing data for each variable
8 summary(dataset)
9
10 #Look at variables for which you have missing data
11 table(dataset$code_birth_place[is.na(dataset$zip_birth_place)])
12
13 #Create useful vars
14 dataset$italian[!is.na(dataset$zip_birth_place)]<-0
15 dataset$italian[is.na(dataset$zip_birth_place)]<-1
16
17 #Define factor variables
18 dataset$canteen_code <- factor(dataset$canteen_code,
19                                levels = c(1,2,3,4,5),
20                                labels = c("Canteen1", "Canteen2", "
21                                           Canteen3", "Canteen4", "Canteen5"))
22
23 dataset$meal_type <- factor(dataset$meal_type,
24                              levels = c(1,2,3,4),
25                              labels = c("Light meal 1", "Light meal 2", "
26                                           Pizza/Sandwich", "Full meal"))
27
28 dataset$on_track <- factor(dataset$on_track,
29                             levels = c(1,2),
30                             labels = c("On track", "Not on track"))
```

```

29
30 dataset$course <- factor(dataset$course,
31                           levels = c(1,2,4,5),
32                           labels = c("Bachelor", "Master degree", "Degree",
33                                       "PhD"))
34
35 dataset$user_type <- factor(dataset$user_type,
36                             levels = c(1,2),
37                             labels = c("Student", "Foreign"))
38
39 dataset$italian <- factor(dataset$italian,
40                           levels = c(1,0),
41                           labels = c("Italian", "Not italian"))
42
43 # Removing and checking duplicate rows and duplicate case
44 dataset_clean <- dataset[!duplicated(dataset), ]
45
46 dataset_clean <- dataset_clean %>%
47   group_by(ID, meal_year) %>%
48   mutate(count_year=n()) %>%
49   ungroup()
50
51 # Remove columns that are not useful or redundant
52 dataset_clean <- dataset_clean %>% select(-c(code_birth_place, zip_birth_place))
53
54 # Remove rows with missing values if appropriate!
55 dataset_clean <- na.omit(dataset_clean)
56
57 # Rename the dataset
58 db_mensa <- dataset_clean
59
60 #Look at data about one year, i.e. 2014
61 db_mensa2014<- db_mensa %>%
62   filter(meal_year=2014)
63
64 #Obtain median calories over accesses
65 db_mensa2014SR <- db_mensa2014 %>%
66   group_by(ID) %>%
67   mutate(calories_median=median(calories)) %>%
68   ungroup()
69
70 #Select single record
71 db_mensa2014SR <- db_mensa2014 %>%
72   distinct(ID, meal_year, .keep_all = TRUE)
73
74 # Remove columns that are not useful or redundant
75 db_mensa2014SR <- db_mensa2014SR %>% select(-c(meal_date, meal_hour, ID, calories,
76         meal_year))
77
78 # Select numeric features to scale data and plot correlation distribution
79 qt_var <- db_mensa2014SR[c("registration_year", "age", "calories_median")]
80
81 # Standardization of numeric variables
82 db_mensa2014SR <- as.data.frame(scale(db_mensa2014SR[, colnames(qt_var)]))
83
84 # Check if everything is ok
85 head(db_mensa2014SR)

```

4.4 Exploratory Data Analysis

Exploratory data analysis helps us understand the structure of the data and spot any potential issues. It is useful to make some a-priori hypothesis on the behavior and the relations concerning the response

variable (outcome) “Meal calories (kcal)”. To this end, we perform a graphic correlation analysis using the R functions `pairs.panels()` and `cor()`, embedded in the corresponding libraries `psych`, and `corrplot`.

Listing 7: EDA: Correlation Plot

```
1 # scatter plot for quantitative variables
2 pairsplot<-pairs.panels(db_mensa[,colnames(qt_var)],
3   method = "pearson", # Correlation function
4   hist.col = "red",
5   density = TRUE, # Show density plots
6   ellipses = TRUE # Correlation ellipses
7 )
```

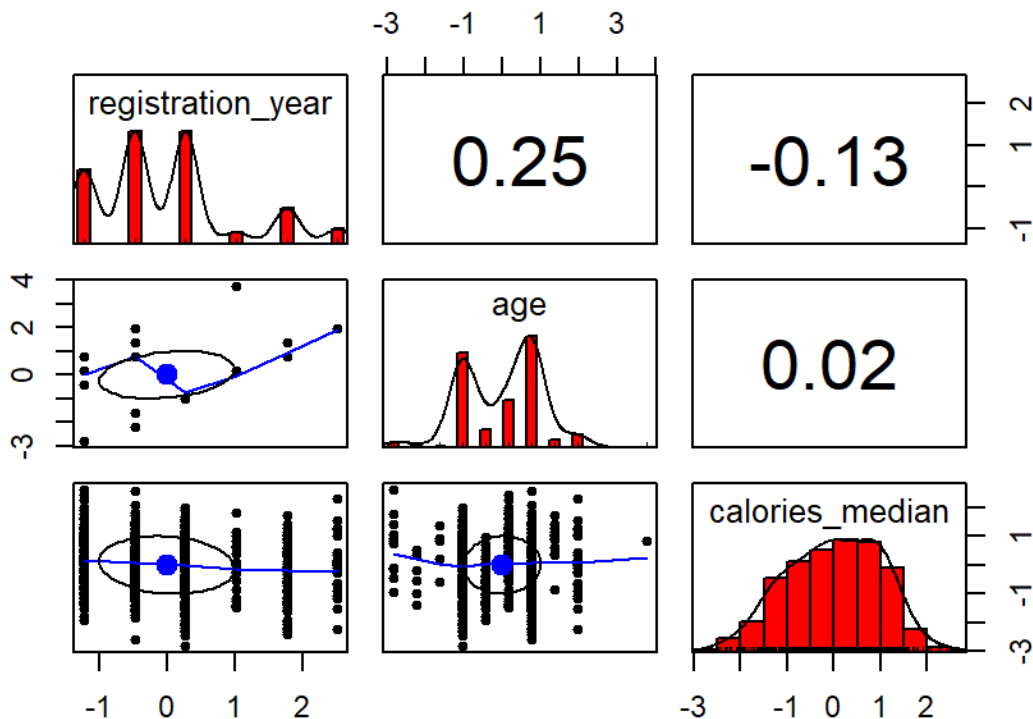


Figure 2: Pair Plot of the database quantitative variables.

The correlation plot in Figure ?? provides a comprehensive visual representation of the relationships between the numerical variables in the dataset. The plot includes scatterplots, correlation coefficients, histograms, and ellipses that illustrate the nature and strength of the relationships between the variables. The scatterplots in the lower-left part of the matrix show the pairwise relationships between the variables, with each point representing a data observation. A red regression line is also included in each scatterplot to highlight the general trend. In the upper-right section of the plot, we see the Pearson correlation coefficients, which provide a numeric measure of the linear relationship between the variables. A correlation value close to 1 suggests a strong positive correlation, meaning that as one variable increases, so does the other. Conversely, a correlation close to -1 indicates a strong negative relationship. The histograms display the distribution of each variable, showing the spread and shape of the data. Overlaid on each histogram is a density curve, which measures whether the data is normally distributed or skewed. Additionally, the scatterplots include correlation ellipses, which visually convey the strength and direction of the correlation between two variables. Narrow ellipses correspond to stronger correlations, while wider ellipses suggest weaker relationships. Ellipses that slope upwards indicate positive correlations,

while downward-sloping ellipses show negative correlations. This correlation plot is a valuable tool for understanding the interdependencies within the dataset and provides a solid foundation before further analysis.